TRUSTDRIVE

Complete Frontend, Backend, AI Services & Deployment Documentation

Project Architecture and Implementation Blueprint

8 November 2026

CONTENTS

1. Executive Summary
2. Problem Statement and Objectives
3. Solution Overview
4. Users, Roles and Permissions
5. Technology Stack
6. System Architecture
7. Recommended Repository Structure
8. Frontend Architecture and Features
9. Backend Architecture and Features
10. API Design and End-to-End Data Flow
11. Database Architecture
12. AI, OCR and Fraud Investigation Services
13. Vehicle Evidence Graph and Trust Assessment
14. 3D Vehicle Inspection Studio
15. Marketplace and Communication Workflows
16. Security, Privacy and Data Integrity
17. Render Deployment Architecture
18. Environment Configuration
19. Testing and Quality Assurance
20. Feature Status and Known Limitations
21. Implementation Roadmap
22. Definition of Done
23. Glossary

Document type: Project architecture and implementation blueprint
Version: 1.0
Project domain: Automation & Digital Transformation
Project: AI-powered multi-category vehicle history verification and intelligent used-vehicle marketplace

> **Accuracy note:** This document is based on the TrustDrive requirements, architecture details and audit findings shared in this conversation. It is a comprehensive reference specification, not a fresh inspection of the source repository. File paths, endpoint names and module boundaries below are recommended logical organization unless explicitly identified as known from the existing project. The status section preserves the latest audit classifications supplied by the project owner. Do not treat a planned feature as implemented merely because it is described here.

1. Executive Summary

TrustDrive is an AI-assisted vehicle history verification and used-vehicle transparency platform. It is designed to help buyers and sellers understand the available history, condition and evidence associated with a vehicle before a transaction.

The platform brings together seller-submitted vehicle information, registration and identity documents, service invoices, inspection records, mileage readings, repair evidence, photographs and other permitted data. It extracts relevant information, compares records, identifies inconsistencies and presents findings in a structured **Vehicle History Passport**.

TrustDrive is not only a vehicle listing website. Its core value is the connection of evidence from multiple sources into one explainable investigation workflow. The marketplace is built around the verification experience.

The platform has three user roles:

- **Seller:** registers vehicles, uploads supporting records, maintains vehicle history, submits inspection evidence and publishes listings.
- **Buyer:** discovers vehicles, reviews available evidence, investigates possible risks, compares vehicles and contacts sellers.
- **Administrator / reviewer:** manages users, review queues, reports, disputes and platform-level controls, according to the permissions implemented by the application.

The system is intended to support petrol and electric cars, motorcycles, scooters and additional vehicle categories through a shared vehicle data model and category-specific forms, checks and 3D assets.

1.1 Product principles

1. **Evidence first:** findings must be tied to records, documents, photographs or clearly identified user submissions.
2. **Explainable outcomes:** users should understand why an inconsistency or risk indicator was raised.
3. **No false certainty:** missing records do not prove that no accident, theft, repair or fraud occurred.
4. **Category-aware verification:** a scooter, motorcycle, electric vehicle and car must not be forced into the same specification rules.
5. **Human review:** AI suggestions are not automatically treated as confirmed physical damage or fraud.
6. **Auditable changes:** material corrections to vehicle history should preserve an audit trail.
7. **Privacy by role:** buyers see only information the seller or platform is permitted to disclose.

2. Problem Statement and Objectives

2.1 Problem

Used-vehicle buyers often cannot reliably understand a vehicle's actual history because information is scattered across registration documents, insurance papers, service invoices, repair bills, inspection reports, odometer photographs and seller statements. These records may be incomplete, inconsistent, difficult to compare or potentially manipulated.

The main problems are:

- **Mileage inconsistency:** an older service record may show a higher distance than a current odometer reading.
- **Hidden damage:** repaired or undisclosed damage may not be apparent from listing photographs.
- **Document inconsistency:** registration, chassis/frame, dates or vehicle details may differ across records.
- **Fragmented history:** buyers must manually collect and compare information from many sources.
- **Unclear provenance:** it can be difficult to know who supplied a record, when it was added and whether the digital file has changed.
- **Vehicle-category differences:** two-wheelers and EVs require different information and checks from petrol cars.

2.2 Objectives

TrustDrive aims to:

- centralize available vehicle records;
- automate document text extraction and field comparison;
- detect chronological and identity inconsistencies;
- organize vehicle history into a readable timeline;
- connect damage records to supporting evidence;
- provide explainable findings and an evidence-based trust assessment;
- let sellers publish more transparent listings;
- help buyers make better-informed decisions;
- support multiple vehicle categories through one extensible platform.

2.3 Scope boundaries

TrustDrive can analyze data it receives or is authorized to access. It must not claim automatic access to government registration, insurance, police or theft databases without an approved integration. A seller-uploaded RC image is not the same as an official real-time government verification.

3. Solution Overview

The central product object is the **Vehicle History Passport**. Each passport groups the records and findings associated with one vehicle.

A typical passport contains:

- normalized vehicle identity and specifications;
- registration and ownership information, where permitted;
- documents and OCR-extracted fields;
- mileage readings with dates and sources;
- service, repair and inspection events;
- accident or damage disclosures;
- photographs and evidence references;
- consistency checks and AI-assisted findings;
- verification status and limitations;
- a versioned, explainable trust assessment;
- a traceable audit history.

3.1 Main product modules

Module	Purpose	Primary users
Authentication	Accounts, sessions and role permissions	All
Seller Studio	Vehicle onboarding and listing management	Seller
Buyer Marketplace	Search, discovery and comparison	Buyer
Document Center	Uploads, OCR and document review	Seller, reviewer
Verification Engine	Identity and record consistency checks	Seller, buyer, reviewer
AI Investigation	Mileage, document and damage-related findings	Seller, buyer, reviewer
Vehicle History Passport	Timeline and evidence summary	Seller, buyer
3D Inspection Studio	Interactive category-specific model and component damage display	Seller, buyer
Admin Console	Review queues, users and platform oversight	Admin
Reports and Notifications	Exports, status updates and communication	All, by permission

4. Users, Roles and Permissions

4.1 Seller

The seller owns or manages a vehicle record and can:

- register and update permitted vehicle details;
- upload documents and photographs;
- submit mileage, service, repair and inspection records;
- review OCR extraction and correct mistakes;
- run available verification workflows;
- review AI findings and provide additional evidence;
- create and manage marketplace listings;
- respond to buyer inquiries;
- submit damage and repair records;
- download seller-accessible reports.

A seller must not silently overwrite immutable or verified history. Where correction is allowed, the system should store the correction as a new audit event with author, timestamp and reason.

4.2 Buyer

The buyer can:

- search and filter published listings;
- view vehicle details and seller disclosures;
- save vehicles and maintain recently viewed history;
- access the disclosed portion of a Vehicle History Passport;
- view permitted mileage and service timelines;
- review suspicious indicators and their supporting evidence;
- inspect supported 3D models and recorded damage hotspots;
- compare vehicles;
- contact sellers and request inspections or test drives;
- generate or download buyer-accessible reports.

A buyer must not access private seller documents, internal moderation notes, private contact information or another user's records unless explicitly permitted.

4.3 Administrator / reviewer

Depending on the actual implementation, administrators may:

- review suspicious or pending records;
- manage accounts and roles;
- inspect platform-level reports;
- review disputed findings;
- manage model asset metadata and supported categories;
- view audit logs;
- disable unsafe accounts or listings;
- configure operational settings.

Administrative functionality must be protected by backend authorization, not just hidden UI navigation.

4.4 Permission matrix

Action	Seller	Buyer	Admin/reviewer
Register own vehicle	Yes	No	As authorized
Edit own draft vehicle	Yes	No	Review/override by policy
Upload own vehicle documents	Yes	No	Review access
View disclosed passport	Own vehicle	Published vehicle	Authorized
View private raw documents	Own / permitted	No by default	Authorized
Publish listing	Own vehicle	No	Moderation control if implemented
Send inquiry	No	Yes	Review if authorized
Reply to inquiry	Own listing	No	Review if authorized

Action	Seller	Buyer	Admin/reviewer
Manage all users	No	No	Yes
Change audit history	No silent edits	No	No silent edits; use audited correction

5. Technology Stack

The stack below reflects the architecture discussed for TrustDrive. Exact package versions must be confirmed from the repository lockfiles and manifests before deployment.

Layer	Technology	Responsibility
Web frontend	Next.js, React, TypeScript	Pages, routing, server rendering and UI
Styling	Tailwind CSS and existing UI components	Responsive design and reusable interface
3D frontend	Three.js, React Three Fiber, Drei	GLB loading, camera controls, mesh picking and damage display
Main application server	Next.js server routes / server-side modules	Authentication and application APIs where currently implemented
Intelligence API	Python, FastAPI	OCR, ML and image-intelligence endpoints
Primary production database	PostgreSQL	Persistent relational application data
Local/demo database	SQLite, where currently used	Development and limited demo environment
ORM/data access	Existing project data layer; SQLAlchemy for Python service if used	Database queries and migrations
OCR	RapidOCR currently reported active; PaddleOCR optional target	Document text extraction
Computer vision	OpenCV and YOLO adapter	Image preprocessing and supported detections
ML	XGBoost / scikit-learn	Experimental or validated anomaly/risk models
LLM orchestration	LangChain / LangGraph when configured	Evidence summarization and investigation orchestration
Background processing	Celery and Redis, if deployed	Long-running OCR/ML/file tasks
Integrity	SHA-256 hashes and audit records	Digital file-integrity checks
Optional blockchain	EVM-compatible network and contract	External anchoring of document hashes, only when implemented
Browser tests	Playwright	Real browser end-to-end checks
Backend tests	Pytest and HTTPX where applicable	Python API and service tests
Frontend tests	Existing test framework / Vitest where configured	Component and logic tests
Deployment	Render or another supported host	Application hosting and managed services
Object storage	S3-compatible storage target	Durable documents, images and reports

5.1 Architectural boundary

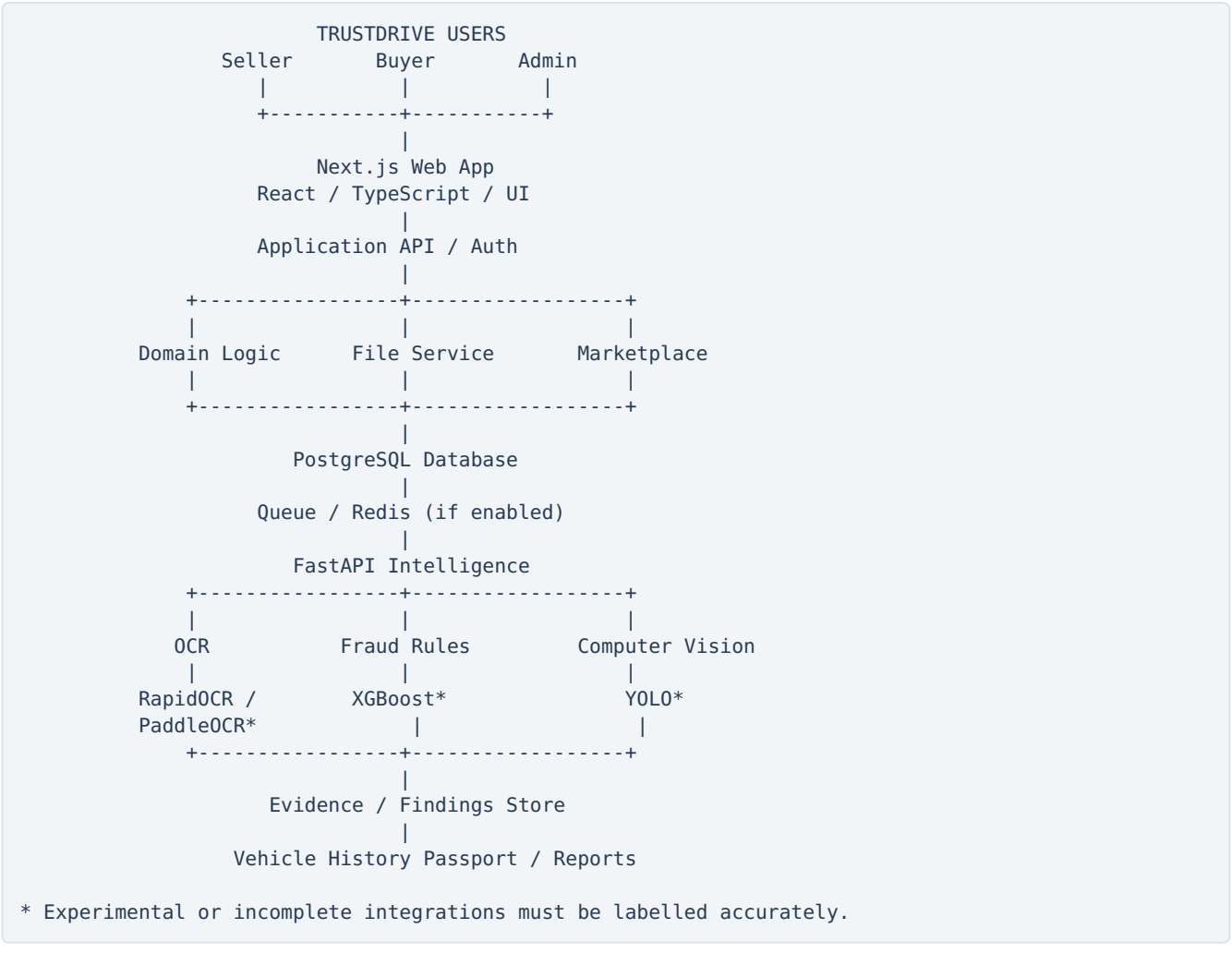
The project context indicates a Next.js application with server-side authentication/database logic and a separate Python FastAPI intelligence service. Keep this separation explicit:

- The **Next.js application** owns user-facing pages and core web application workflows.
- The **FastAPI intelligence service** owns Python-based OCR, computer vision and ML operations.
- A **worker** performs long-running tasks where a queue is configured.
- The **database** stores users, vehicles, events, verification results and metadata.
- **Object storage** stores file bytes; the database stores metadata and secure object references.

Do not move Next.js API routes or server-only modules into a separate Python backend merely to make folder names look uniform. Preserve runtime dependencies.

6. System Architecture

6.1 Logical architecture



6.2 Main request lifecycle

1. A user performs an action in the browser.
2. The frontend validates basic input and submits a request.
3. The application API authenticates the user and checks role/object permissions.
4. The backend validates the request and creates or updates a domain record.
5. If the action requires OCR, image processing or another long-running task, the backend creates a processing job.
6. The FastAPI service or worker processes the job using configured providers/models.
7. Results are normalized and saved with source, status, model/provider version and timestamps.
8. The API returns the result or a job identifier/status.
9. The frontend refreshes the affected view and displays the result, evidence and any limitations.
10. Audit events record material changes.

6.3 Data flow principle

The UI must never claim success solely because a button was clicked. A successful workflow requires a valid backend response and, for persistent operations, confirmation that the data was saved and can be retrieved again.

7. Recommended Repository Structure

This is a logical target structure. Keep files in locations required by Next.js, Python packaging, Render, Docker and existing import paths.

```
TrustDrive/
├── frontend/
│   ├── app/                # Next.js routes and pages
│   ├── components/         # Reusable UI components
│   ├── hooks/              # React hooks
│   ├── lib/                # Client and server utilities
│   │   ├── server/         # Auth, DB and server-only logic
│   │   └── api/            # API clients
│   ├── public/             # Images, icons and public GLB assets
│   ├── styles/             # Global and feature styles
│   ├── package.json
│   └── lockfile
├── backend/                # Python FastAPI application, if separated
│   ├── app/
│   ├── api/
│   ├── models/
│   ├── schemas/
│   ├── repositories/
│   ├── services/
│   ├── migrations/
│   └── requirements.txt
├── services/
│   ├── ai/
│   │   ├── ocr/
│   │   ├── fraud_detection/
│   │   ├── computer_vision/
│   │   ├── yolo/
│   │   ├── xgboost/
│   │   └── llm/
│   └── workers/
├── tests/
│   ├── frontend/
│   ├── backend/
│   ├── ai/
│   ├── integration/
│   ├── e2e/
│   └── security/
├── project/
│   ├── documentation/
│   ├── reports/
│   ├── scripts/
│   └── fixtures/
├── infra/
│   ├── docker/
│   └── deployment/
├── .github/workflows/
├── render.yaml
├── .env.example
├── .gitignore
├── README.md
└── LICENSE
```

Folder organization rules

- Keep frontend code required by Next.js inside its application root.
- Keep Python dependencies and package configuration with the Python application.
- Keep large model weights and private uploads out of Git unless licensing and repository size permit; use storage or release artifacts as appropriate.
- Keep tests in a predictable test tree, while preserving framework-required config files.
- Keep `render.yaml`, root workspace files and CI configuration where the tooling expects them.
- Never move files without updating imports, Docker contexts, build commands, tests and deployment configuration.

8. Frontend Architecture and Features

The frontend should be organized around role-based layouts and reusable feature modules. The following is the complete functional specification for the interface.

8.1 Public landing page

Purpose: explain TrustDrive and direct visitors to registration, login or vehicle discovery.

Recommended sections:

- Hero with project value proposition and primary actions.
- How TrustDrive works.
- Vehicle categories supported.
- Vehicle History Passport explanation.
- Document and mileage verification explanation.
- 3D inspection preview.
- Buyer and seller benefits.
- Trust/safety limitations and FAQs.
- Footer with contact, privacy and terms.

Functional requirements:

- Login and registration links must work.
- Marketplace links must route correctly.
- Marketing statistics must not be fabricated.
- Do not imply official government integrations unless active.

8.2 Authentication pages

Features:

- User registration.
- Login.
- Role selection where supported.
- Password validation.
- Session establishment.
- Logout.
- Protected route redirection.
- Password reset/email verification only if configured.
- Demo login only in a deliberately configured demo environment.

Frontend responsibility:

- Validate required fields.
- Display actionable errors.
- Avoid storing plaintext passwords.
- Never trust a client-provided role as authorization.

Backend responsibility:

- Hash passwords with a suitable password hashing scheme.

- Verify credentials.
- Issue and validate secure sessions/tokens.
- Apply rate limiting and role authorization.
- Prevent user enumeration through overly specific login errors.

8.3 Seller dashboard

Overview

Display metrics derived from actual database queries:

- Total registered vehicles.
- Active listings.
- Pending verification.
- Verified vehicles.
- Alerts requiring review.
- Buyer inquiries.
- Listing views, if tracked.
- Saved/favourited counts, if available.

Dashboard charts and recent activity must use real persisted records, not constants.

My Vehicles

Seller actions:

- Add a vehicle.
- View vehicle profile.
- Edit permitted fields.
- Upload photographs.
- Upload documents.
- Add mileage and history events.
- Open verification results.
- Open 3D inspection.
- Create a listing.
- Pause or mark a listing sold.

Vehicle onboarding wizard

Suggested steps:

1. Category and powertrain.
2. Identity and specifications.
3. Photographs.
4. Documents.
5. Mileage and history.
6. Verification.
7. Review and save.
8. Optional listing publication.

Common fields:

- Category/subcategory.

- Manufacturer, model, variant and year.
- Registration number where applicable.
- VIN/chassis/frame identifier as applicable.
- Colour, location, ownership and condition.
- Asking price where listing is intended.

Category-specific fields:

- Petrol vehicle: fuel type, engine capacity/number where relevant, transmission and odometer.
- Electric vehicle: motor information where available, battery capacity/chemistry/serial where available, charging details, battery warranty/replacement history and total odometer distance.
- Two-wheeler: frame/chassis data and category-specific component condition.
- Commercial vehicle: permitted body type, usage and carrying specification fields.

Do not force a 17-character VIN rule on every two-wheeler or market.

Documents page

The seller should be able to:

- select a vehicle;
- upload supported files;
- see upload and scan status;
- see OCR processing status;
- review extracted fields;
- correct extracted values where allowed;
- resubmit a corrected document;
- view verification findings;
- download their own permitted files.

Suggested statuses: `UPLOADED`, `SCANNING`, `PROCESSING`, `EXTRACTED`, `NEEDS_REVIEW`, `VERIFIED`, `SUSPICIOUS`, `REJECTED`, `FAILED`.

Vehicle history page

A chronological timeline for:

- registration and ownership events;
- service records;
- repair records;
- mileage readings;
- accident/damage events;
- inspection events;
- insurance/PUC records where provided;
- document upload and verification events.

Each event should include date, source type, creator, evidence references, mileage (where relevant), status and audit metadata.

Verification Center

Show separate checks for:

- vehicle identity matching;
- registration field consistency;

- chassis/frame/VIN matching where applicable;
- document duplicate detection;
- date consistency;
- mileage chronology;
- source/provenance status;
- document integrity hash;
- external provider status, only if connected.

AI Fraud Analysis

The page should show:

- finding title;
- severity;
- affected vehicle/document/event;
- reason for the finding;
- evidence references;
- rule/model/provider that generated it;
- confidence or score only where properly calibrated;
- review status;
- recommended next step.

AI suggestions must be distinguishable from confirmed facts.

Seller 3D Studio

The seller can open a supported model, rotate/zoom, select mapped components and add a damage/inspection record. Save must create a backend record and attach evidence files. Do not allow unsupported 3D controls to appear as working.

Marketplace management

- Create listing from an eligible vehicle.
- Add photos and description.
- Set price and location.
- Preview listing.
- Publish, pause, resume or mark sold.
- Track inquiries and listing performance if events are recorded.

Seller inquiries and notifications

- List buyer inquiries.
- View message thread.
- Reply.
- Update inquiry state.
- Review test-drive/inspection requests.
- Receive in-app notifications.
- Email/SMS only when a provider is configured and delivery succeeds.

8.4 Buyer dashboard

Overview

Metrics may include:

- Vehicles explored.
- Saved vehicles.
- Recently viewed vehicles.
- Active inquiries.
- Reports generated.
- Recommended vehicles, if recommendation logic exists.

Marketplace discovery

Search and filter by supported fields:

- category and powertrain;
- manufacturer/model;
- price range;
- year;
- distance/mileage;
- location;
- ownership;
- condition;
- verification status;
- available history/damage disclosure.

Sorting should be deterministic and pagination should be server-side for larger result sets.

Vehicle detail

Show:

- listing photographs and specifications;
- seller disclosures;
- price and location;
- verification summary;
- Vehicle History Passport entry point;
- mileage timeline;
- service/repair history where disclosed;
- recorded damage findings;
- model accuracy label for 3D display;
- contact/inquiry action.

Vehicle History Passport

The buyer sees a permission-filtered version of the vehicle's history. It should clearly identify:

- confirmed information;
- seller-submitted information;
- AI/rule-generated findings;
- records awaiting review;

- unavailable external information;
- evidence that cannot be disclosed.

Compare, wishlist and recently viewed

- Save/remove a vehicle.
- Persist wishlist records per user.
- Compare supported attributes side by side.
- Show unavailable values as unknown rather than filling them with guessed values.

Buyer communication

- Send inquiry.
- Request test drive.
- Request inspection.
- See response/status.
- Receive in-app notifications.

Buyer reports

Generate a report from persisted vehicle information and findings. Include a generation date, vehicle identifier, sources, limitations and verification status. Do not describe missing data as a clean history.

8.5 Admin console

Only include routes and controls actually implemented. A complete admin module may include:

- user management;
- listing moderation;
- pending verification queue;
- disputed findings;
- document review;
- model/asset registry administration;
- audit log viewer;
- system health and provider status.

All admin actions must be backend-authorized and audited.

8.6 Frontend component standards

- Use reusable form controls and field validation.
- Show loading, empty, success, partial and error states.
- Prevent duplicate submissions.
- Keep API errors readable and specific enough to help the user.
- Ensure keyboard navigation and accessible labels.
- Test responsive layouts.
- Do not show success until the server confirms the operation.
- Avoid hardcoded dashboard metrics and demo-only records in production mode.

9. Backend Architecture and Features

The backend is responsible for authentication, authorization, business rules, data persistence, file handling, orchestration and provider integration.

9.1 Core backend modules

Module	Responsibilities
Authentication	Registration, login, logout, session/token verification
User/RBAC	Profiles, roles, user-level permissions and data isolation
Vehicle	Vehicle creation, updates, identifiers, category validation
Document	Upload metadata, storage references, type and access validation
OCR orchestration	Create jobs, send files to OCR provider, save extracted results
History	Chronological service, repair, mileage and ownership events
Verification	Rules for identity, dates, fields and duplicate evidence
Investigation	Combine evidence, findings and review states
Damage	Damage records, component mapping, repair details and photos
Listing	Draft, publish, pause, resume and sold states
Inquiry	Buyer/seller messages and inspection requests
Report	Generate permission-filtered PDF/JSON reports
Notification	In-app events and optional email/SMS provider
Audit	Record material changes and privileged operations
Asset registry	3D model metadata, license, categories and capabilities

9.2 Request validation

Every write endpoint should validate:

- authenticated user;
- role and object ownership;
- required fields;
- category-specific constraints;
- file type and size;
- allowed state transition;
- foreign key references;
- duplicate identifiers according to market rules;
- rate limits and abuse conditions.

The backend, not the browser, is the final authority for validation and permissions.

9.3 Background jobs

Long-running tasks should not block a web request. Suitable jobs include:

- OCR on multi-page PDF documents;
- large image processing;
- YOLO inference;
- batch history analysis;
- report generation;
- email delivery;
- file scanning.

A job should have an ID, type, input references, status, attempt count, created/started/completed timestamps, result reference and safe error details. Retry only transient failures and make jobs idempotent where possible.

Suggested job states: `QUEUED`, `RUNNING`, `SUCCEEDED`, `FAILED`, `CANCELLED`, `NEEDS_REVIEW`.

10. API Design and End-to-End Data Flow

The exact endpoint paths must be verified against the repository. The following is a recommended API catalogue and naming convention, not a claim that every route currently exists.

10.1 Authentication API examples

Method	Suggested route	Purpose
POST	`/api/v1/auth/register`	Register account
POST	`/api/v1/auth/login`	Authenticate user
POST	`/api/v1/auth/logout`	End session
GET	`/api/v1/auth/me`	Return current user and role

10.2 Vehicle API examples

Method	Suggested route	Purpose
GET	`/api/v1/vehicles`	List permitted vehicles
POST	`/api/v1/vehicles`	Create vehicle
GET	`/api/v1/vehicles/{id}`	Get vehicle detail
PATCH	`/api/v1/vehicles/{id}`	Update permitted fields
POST	`/api/v1/vehicles/{id}/archive`	Archive vehicle
GET	`/api/v1/vehicles/{id}/history`	Get history timeline

10.3 Documents and OCR API examples

Method	Suggested route	Purpose
POST	`/api/v1/vehicles/{id}/documents`	Upload document
GET	`/api/v1/documents/{id}`	Read permitted metadata/status
GET	`/api/v1/documents/{id}/download`	Secure file access
POST	`/api/v1/documents/{id}/process`	Start OCR/verification job
GET	`/api/v1/jobs/{jobId}`	Poll processing status
PATCH	`/api/v1/documents/{id}/extracted-fields`	Review/correct extracted fields

10.4 Verification and investigation API examples

Method	Suggested route	Purpose
POST	`/api/v1/vehicles/{id}/verify`	Run deterministic checks
GET	`/api/v1/vehicles/{id}/verification`	Read verification results
POST	`/api/v1/vehicles/{id}/investigations`	Start evidence investigation
GET	`/api/v1/investigations/{id}`	Read findings and status
POST	`/api/v1/findings/{id}/review`	Record authorized human review

10.5 Marketplace and buyer API examples

Method	Suggested route	Purpose
GET	`/api/v1/listings`	Search published listings
POST	`/api/v1/vehicles/{id}/listings`	Create listing
PATCH	`/api/v1/listings/{id}`	Update listing
POST	`/api/v1/listings/{id}/publish`	Publish listing
POST	`/api/v1/listings/{id}/pause`	Pause listing
POST	`/api/v1/listings/{id}/sold`	Mark sold
POST	`/api/v1/listings/{id}/inquiries`	Create inquiry
GET	`/api/v1/me/wishlist`	Get buyer wishlist
POST	`/api/v1/me/wishlist/{vehicleId}`	Save vehicle
DELETE	`/api/v1/me/wishlist/{vehicleId}`	Remove saved vehicle

10.6 3D and damage API examples

Method	Suggested route	Purpose
GET	`/api/v1/vehicle-models`	List available model metadata
GET	`/api/v1/vehicles/{id}/3d-model`	Resolve compatible model

Method	Suggested route	Purpose
GET	`/api/v1/vehicles/{id}/damage`	Get permitted damage records
POST	`/api/v1/vehicles/{id}/damage`	Add damage/inspection record
PATCH	`/api/v1/damage/{damageId}`	Update permitted damage record
POST	`/api/v1/damage/{damageId}/evidence`	Attach photo/document evidence

10.7 Standard API response pattern

Use a consistent response envelope where suitable:

```
{
  "data": {},
  "meta": {
    "requestId": "request-id",
    "timestamp": "ISO-8601 timestamp"
  },
  "error": null
}
```

For an error:

```
{
  "data": null,
  "meta": {"requestId": "request-id"},
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "A required field is missing",
    "fields": {"registrationNumber": "Required"}
  }
}
```

Do not expose stack traces, secret paths, database credentials or private document URLs in API errors.

10.8 Example end-to-end: add a vehicle

1. Seller completes the Add Vehicle wizard.
2. Frontend sends the payload to the application API.
3. Backend checks session, seller role and field validation.
4. Backend normalizes category, powertrain and identifiers.
5. Backend checks permitted duplicate rules.
6. Backend inserts vehicle and category-specific specification records.
7. Backend writes an audit event.
8. API returns the saved vehicle ID and normalized data.
9. Frontend navigates to the vehicle profile.
10. Seller refreshes the page; the same record is retrieved from the database.

10.9 Example end-to-end: upload and process a document

1. Seller selects a document type and file.
2. Backend validates the request and ownership of the vehicle.

3. File is quarantined or stored in a controlled upload area.
4. Antivirus scan runs if configured.
5. File metadata and secure object reference are persisted.
6. OCR job is queued.
7. OCR provider extracts text and fields.
8. Normalizer converts dates, identifiers and mileage into consistent formats.
9. Verification rules compare extracted fields with the vehicle record and other evidence.
10. Results and confidence/provenance metadata are saved.
11. Seller reviews the extracted fields and corrects them if necessary.
12. The passport displays the document status and associated findings.

10.10 Example end-to-end: buyer investigation

1. Buyer searches marketplace.
2. API returns published listings and filters.
3. Buyer opens a vehicle.
4. Backend returns only public listing fields and permission-allowed history.
5. Buyer opens the passport.
6. Investigation service returns findings with evidence references and limitations.
7. Buyer opens a finding to see its source record or permitted evidence.
8. Buyer sends an inquiry.
9. Backend saves the inquiry and creates an in-app notification for the seller.
10. Seller replies; buyer sees updated conversation status.

11. Database Architecture

Use a relational database for core records. PostgreSQL is the recommended production target for multi-service deployment. SQLite may remain useful for local development or a constrained demo, but production persistence must be deliberately configured.

11.1 Main entities

Entity	Main purpose	Important relationships
users	Accounts and identity	Roles, vehicles, inquiries, notifications
roles / permissions	Access control	Users and permissions
vehicles	Common vehicle identity	Seller, specifications, documents, history, listing
vehicle_specs	Category-specific details	Vehicle
documents	File metadata and type	Vehicle, uploader, OCR results
document_extractions	OCR text/fields and confidence	Document, provider/model version
mileage_records	Dated readings	Vehicle, source document/event
history_events	Service/repair/ownership/inspection timeline	Vehicle, actor, evidence
damage_records	Component damage and repair status	Vehicle, component, inspection
damage_evidence	Photos and file references	Damage record, document/object storage

Entity	Main purpose	Important relationships
inspections	Inspection session and result	Vehicle, inspector, findings
verification_runs	Rules and results	Vehicle, input evidence, findings
findings	Suspicious/inconsistent observations	Vehicle, verification run, evidence
trust_assessments	Versioned explainable assessment	Vehicle, evidence snapshot, rule/model version
listings	Marketplace publication	Vehicle, seller
wishlists	Buyer saved vehicles	Buyer, vehicle
inquiries	Buyer/seller communication	Buyer, seller, listing
notifications	In-app and provider delivery status	User, event
audit_events	Change history	Actor, entity, event type
model_assets	3D model registry	Category, manufacturer/model mapping
model_components	Mesh-to-component mapping	Model asset
background_jobs	Async processing status	Source entity, provider result

Names are logical entities. Confirm actual table names and schema from migrations before applying this design.

11.2 Common vehicle record

The core vehicle entity should contain common information such as:

- unique internal ID;
- seller/owner account reference;
- category and subcategory;
- manufacturer, model, variant and year;
- registration identifier where applicable;
- VIN/chassis/frame identifier where applicable;
- powertrain/fuel type;
- location and status;
- created/updated timestamps.

Keep category-specific details in normalized specification tables or a carefully validated JSON column. Do not put every possible car, motorcycle and EV field into mandatory columns.

11.3 Mileage records

A mileage record should include:

- vehicle ID;
- numeric distance;
- unit;
- record date;
- source type;
- source document or inspection reference;
- extracted/manual flag;
- confidence/review status;
- creator and timestamp.

All readings must be normalized to a consistent unit before chronology checks. A decrease should trigger review, not automatic proof of fraud.

11.4 Damage records

A damage record should contain:

- vehicle ID;
- logical component ID;
- damage type;
- severity;
- description;
- reported and inspection dates;
- repair status;
- repair details;
- evidence references;
- author and review status.

Keep damage reports separate from raw AI detections. A model prediction should become a human-reviewed damage record only through an explicit review step.

11.5 Database integrity rules

- Use foreign keys for related entities.
- Add unique constraints only for identifiers whose uniqueness rules are valid in the target jurisdiction/category.
- Use transactions for multi-record workflows.
- Use migrations instead of manual production schema changes.
- Use backups before migrations.
- Use soft deletion or archive states where records must remain auditable.
- Store file bytes in object storage, not directly in a relational row unless a deliberate small-file use case exists.

12. AI, OCR and Fraud Investigation Services

The AI system should be modular. A provider can be disabled or replaced without breaking the deterministic verification engine.

12.1 OCR service

Intended workflow

`Document → preprocessing → OCR text → field extraction → normalization → comparison → review → persistence`

OCR can extract:

- registration numbers;
- chassis/frame/VIN identifiers where visible;
- engine numbers where applicable;
- dates;
- mileage values;
- names and addresses where legally appropriate;
- document references and policy/invoice fields.

OCR output is not proof of document authenticity. It only describes what the OCR system read from the image or PDF.

Current audit status

- RapidOCR was reported as active.
- PaddleOCR was reported as not installed.
- Therefore OCR should not be described as wholly unavailable if RapidOCR upload-to-result flow is actually passing tests.
- PaddleOCR remains a separate provider/upgrade until installed, configured and tested.

OCR quality checks

- character/field accuracy on a representative test set;
- confidence calibration or useful confidence indicators;
- date and number normalization;
- unreadable image handling;
- page handling for PDFs;
- duplicate document checks;
- manual correction flow;
- provider/version tracking.

12.2 Deterministic verification rules

Deterministic rules are suitable for checks with explicit logic, including:

- mileage readings that decrease over time;
- incompatible registration or chassis values across records;
- duplicate file hashes;
- duplicate document references;
- invalid date sequences;
- conflicting model/year fields;
- missing required category-specific data;
- a document linked to a different vehicle identifier.

A rule should return:

- rule ID and version;
- input records;
- result;
- severity;
- explanation;

- evidence references;
- review state.

These rules are often more dependable than an LLM for exact comparisons.

12.3 XGBoost model

XGBoost may learn patterns from labelled historical examples, but a model trained only on synthetic demo rows is not production validated.

A real production pipeline requires:

1. authorized and representative labelled dataset;
2. documented feature definitions;
3. removal of data leakage;
4. train/validation/test split;
5. class imbalance analysis;
6. precision, recall, F1 and confusion matrix;
7. threshold selection based on false-positive costs;
8. model/version registry;
9. drift and quality monitoring;
10. human review for high-impact findings.

Until this exists, label XGBoost `DEMO_ONLY` or `EXPERIMENTAL`, not production.

12.4 YOLO / image damage detection

Expected flow:

`Photo upload → file validation → safe storage/scan → image preprocessing → model inference → bounding boxes/classes/confidence → seller review → confirmed damage record`

The audit reported that an adapter/weights exist but the complete upload-to-detection flow was not implemented/tested. Do not claim complete YOLO damage detection until that full workflow is connected and tested with suitable vehicle-damage weights.

A detection is a visual suggestion, not a definitive mechanical inspection. Lighting, angle, dirt, reflections and repair quality can cause false detections or missed damage.

12.5 LLM narrative and LangGraph

LLMs may summarize structured findings, explain the evidence timeline and produce a readable investigation narrative. They must not invent facts or alter the source-of-truth records.

Recommended division:

- deterministic rules: identity, date and numeric comparisons;
- ML: supported anomaly/classification tasks with validated datasets;
- LLM: narrative summary and user-friendly explanation;
- human reviewer: resolves disputed or uncertain findings.

LangGraph can orchestrate multi-step workflows where there are real tool calls, state transitions and human review. Do not add agents solely for appearance.

Current audit status

`TD_LLM_PROVIDER=none` means provider-backed LLM narrative is disabled. Deterministic evidence-only reports can still operate. The LLM feature should remain disabled or clearly labelled until a supported provider is configured and its live call is tested.

12.6 AI output requirements

Every AI result should preserve:

- provider/model name and version;
- job ID and timestamps;
- input evidence references;
- output schema;
- confidence where meaningful;
- status and error information;
- human review state;
- reproducibility metadata where possible.

Never present a model-generated trust score as an official safety certificate.

13. Vehicle Evidence Graph and Trust Assessment

13.1 Evidence graph concept

An evidence graph connects entities and observations that refer to the same vehicle.

Example nodes:

- Vehicle;
- Document;
- Mileage reading;
- Service event;
- Repair event;
- Damage report;
- Inspection;
- Seller statement;
- Verification finding.

Example relationships:

- document belongs to vehicle;
- invoice reports mileage;
- inspection observed component damage;
- service event occurred on date;
- finding was generated from two conflicting records.

The graph can be implemented through relational foreign keys and join queries first. A dedicated graph database is not required for an initial product.

13.2 Trust assessment

A trust assessment should be transparent and versioned. It may summarize:

- identity consistency;
- document completeness;
- mileage chronology;
- history coverage;
- unresolved findings;
- inspection evidence;
- source quality.

The assessment must not:

- imply government approval without a real source;
- treat missing information as proof of fraud;
- claim a physical inspection occurred when it did not;
- hide unresolved or conflicting evidence;
- assign arbitrary scores from an LLM.

A safer output may use clear statuses such as `VERIFIED`, `PENDING`, `SUSPICIOUS`, `UNVERIFIED`, `NEEDS_REVIEW` and `UNAVAILABLE`, with explanations. If a numeric score is used, document its formula, version, input coverage and limitations.

14. 3D Vehicle Inspection Studio

The 3D studio is a visualization layer connected to actual inspection and damage records. It does not independently scan or prove the physical condition of the user's vehicle.

14.1 Core technology

- Three.js for WebGL rendering;
- React Three Fiber for React integration, if used;
- Drei utilities for controls and loaders, if used;
- GLTFLoader for glTF/GLB assets;
- Raycasting / mesh pointer events for selection;
- GLB files for models, materials and textures.

14.2 Asset registry

Each model asset should record:

- asset ID;
- vehicle category and subtype;
- manufacturer/model/year compatibility if known;

- GLB path;
- source URL;
- individual license and attribution;
- unit scale and orientation;
- preview image;
- mesh hierarchy;
- component mapping;
- capabilities supported;
- accuracy designation.

Model selection priority:

1. Exact matching model with verified compatibility.
2. Compatible same-model-family asset.
3. Category representative.
4. Category-specific schematic.
5. Clear unavailable state.

Never display a car model for a motorcycle or scooter.

14.3 Component mapping

A model component registry maps a stable component ID to one or more actual mesh names. For example:

- `front-wheel`;
- `rear-wheel`;
- `fuel-tank`;
- `front-fairing`;
- `scooter-front-apron`;
- `battery-enclosure` only if present in the model;
- `front-bumper`;
- `left-front-door`.

Do not claim a component can be selected if the GLB contains only one merged mesh or has no mapping for that component.

14.4 Interactions

Interaction	Requirement
Orbit / rotation	Rotate around a stable vehicle target
Zoom	Keep model within useful camera bounds
Pan	Move camera target with limits
Preset views	Front, rear, left, right, top and perspective where supported
Component pick	Raycast actual selectable mesh
Highlight	Preserve original material and restore after selection
Damage hotspot	Place marker using local model/component coordinates where available
X-Ray	Temporarily change material rendering and restore it afterwards
Exploded view	Only when model has real separate components and displacement metadata

Interaction	Requirement
Reset	Restore original transforms, materials and camera

14.5 Damage workflow

Seller selects a component → enters damage type/severity/date/description → attaches photographs → submits → backend stores record → viewer renders hotspot → buyer sees permitted finding and evidence.

The hotspot should refer to a stable damage record ID, not random client-side state. If location is approximate, label it approximate.

14.6 Current audit status

The latest audit supplied for this document says:

- Cars currently use a labelled generic GLB.
- Two-wheelers currently use a labelled schematic.
- Licensed non-car GLBs are not implemented.
- Vehicle-specific 3D scan geometry is not implemented.

Therefore, the current viewer should not be described as exact vehicle-level 3D inspection for all categories. The immediate improvement is to add properly licensed category-specific models and honest component capability metadata.

15. Marketplace and Communication Workflows

15.1 Listing lifecycle

Suggested listing states:

```
DRAFT → PENDING_REVIEW → PUBLISHED → PAUSED → SOLD / ARCHIVED
```

Allowed transitions must be validated server-side. A vehicle should not become publicly listed simply because the frontend changed its local status.

15.2 Inquiry lifecycle

Suggested inquiry states:

```
NEW → OPEN → RESPONDED → INSPECTION_REQUESTED / TEST_DRIVE_REQUESTED → CLOSED
```

Keep message access restricted to the participants and authorized moderators.

15.3 Listing quality

A listing should clearly show:

- seller-provided description;
- vehicle details;
- asking price;
- photographs;
- location;
- ownership disclosure;
- available verification status;
- known defects/damage disclosed by seller;
- link to the permitted passport summary.

Do not use a verified badge when only partial user-submitted evidence has been checked.

16. Security, Privacy and Data Integrity

16.1 Authentication

Use a secure password hashing function and session/token validation. The reported project implementation uses Node.js crypto scrypt and JWT-based sessions; confirm current code before relying on these exact details.

Security requirements:

- no plaintext passwords;
- secure secret storage;
- HTTPS in production;
- cookie protections where cookie sessions are used;
- expiration and revocation handling;
- login throttling;
- role checks on every protected API;
- safe error messages;
- no secret values in logs.

16.2 Authorization

Check both role and record ownership. A Seller must not access another Seller's private vehicle/document by changing an ID in a URL. A Buyer must not access raw private documents unless explicitly disclosed and allowed.

16.3 File security

- allow-list supported file types;
- verify actual file content, not just extension;
- enforce size and page limits;

- store in quarantine before scanning where configured;
- use safe random storage keys;
- avoid public bucket access for private files;
- use short-lived signed URLs;
- log file access where appropriate;
- never render untrusted HTML or execute uploaded files.

16.4 Integrity

SHA-256 can detect whether a file's bytes changed relative to a previously stored hash. It does not prove that the original document was genuine, that the issuer created it, or that the information is true.

A local hash chain is not blockchain. Blockchain anchoring requires an actual transaction on a network and retrieval/verification of the stored hash.

16.5 Auditability

Material changes should record actor, entity, action, time, before/after values where appropriate, reason and request/job ID. Sensitive fields and secrets should not be copied into audit logs.

17. Render Deployment Architecture

Render deployment should match the actual runtime boundaries. A practical target may include:

1. **Next.js Web Service:** frontend and any Next.js server-side application APIs.
2. **FastAPI Web Service:** Python intelligence endpoints, if deployed separately.
3. **Background Worker:** Celery/worker process for long-running tasks, if configured.
4. **PostgreSQL:** persistent relational data.
5. **Redis/Key Value:** queue/cache if required.
6. **Object storage:** S3-compatible bucket for uploaded documents, photos and reports.

Not every module must be a separate Render service. Avoid splitting services when that introduces unnecessary cost and operational complexity.

17.1 Deployment preparation

Before deployment:

- confirm repository root and each service root directory;
- verify build and start commands from actual package scripts;
- provide production environment variables through Render;
- run database migrations safely;
- configure persistent storage for files or use object storage;
- configure health and readiness checks;
- configure internal service URLs;
- verify worker start command;
- prevent demo seeding and anonymous workspace access in production;
- configure logs and basic monitoring;

- test a clean redeploy and restart.

17.2 Persistent data warning

Do not store important production database files or uploaded documents only in an ephemeral container directory such as `/tmp`. Use managed PostgreSQL and persistent object storage, or a properly configured persistent disk where supported. Back up data before migrating from SQLite.

17.3 Render deployment checklist

- [ ] Frontend build succeeds.
- [ ] Backend starts on the assigned host and port.
- [ ] Health endpoint succeeds.
- [ ] Database connection succeeds.
- [ ] Migrations complete.
- [ ] File storage survives restart.
- [ ] Worker can reach queue and services.
- [ ] CORS and internal URLs are correct.
- [ ] Production secrets are configured.
- [ ] Demo seeding is disabled in production.
- [ ] Anonymous access is disabled in production.
- [ ] Buyer and Seller logins work with real test accounts.
- [ ] Logs contain no passwords or secrets.
- [ ] Backup and rollback procedures exist.

18. Environment Configuration

The following is a template only. Use the exact variable names expected by the code. Do not paste real secret values into this document or commit `.env` files.

```
# Runtime
NODE_ENV=production
PORT=10000

# Authentication
JWT_SECRET=REPLACE_WITH_LONG_RANDOM_SECRET
ADMIN_EMAIL=REPLACE_WITH_ADMIN_EMAIL
ADMIN_PASSWORD=REPLACE_WITH_STRONG_SECRET
TD_SEED_DEMO=0

# Database
DATABASE_URL=REPLACE_WITH_MANAGED_POSTGRES_URL
TD_DB_FILE=REPLACE_WITH_LOCAL_SQLITE_PATH_IF_USED

# Intelligence API
TD_INTELLIGENCE_URL=REPLACE_WITH_INTERNAL_FASTAPI_URL

# OCR
OCR_PROVIDER=rapidocr
# PaddleOCR settings only after installation/configuration

# LLM
TD_LLM_PROVIDER=none
LLM_API_KEY=
LLM_MODEL=

# Object storage
STORAGE_PROVIDER=local
S3_ENDPOINT_URL=
S3_BUCKET=
S3_REGION=
S3_ACCESS_KEY_ID=
S3_SECRET_ACCESS_KEY=

# Queue
REDIS_URL=

# Malware scanner
CLAMAV_HOST=
CLAMAV_PORT=

# Email / SMS
SMTP_HOST=
SMTP_PORT=
SMTP_USERNAME=
SMTP_PASSWORD=
SMS_PROVIDER=
SMS_API_KEY=

# Blockchain (optional, only when implemented)
BLOCKCHAIN_RPC_URL=
BLOCKCHAIN_CHAIN_ID=
BLOCKCHAIN_CONTRACT_ADDRESS=
BLOCKCHAIN_SIGNING_KEY=
```

Environment safety rules

- Never use a demo password as a production administrator password.
- Never expose database, storage, JWT or wallet secrets in frontend variables.
- Avoid `NEXT_PUBLIC_` prefixes for private credentials.
- Use separate credentials for development, staging and production.
- Rotate secrets if they are accidentally committed or exposed.
- Keep `TD_SEED_DEMO=0` for production and verify anonymous demo fallback is disabled.

19. Testing and Quality Assurance

A feature is complete only when the complete workflow works through the UI and persists correctly.

19.1 Testing layers

Layer	Tool / approach	Purpose
Unit	Existing JS/Python test tools	Small functions and validation rules
Frontend component	React Testing Library / existing framework	UI behaviour
Backend API	Pytest, HTTPX or existing server tests	Request validation and business logic
Database integration	Isolated test DB	Persistence, migrations and relations
AI evaluation	Fixed labelled test fixtures	OCR/detection/model quality
End-to-end	Playwright	Real browser workflows
Security	RBAC and upload tests	Cross-user access and file safety
Deployment	CI build and health checks	Reproducible build and startup

19.2 Required end-to-end journeys

Journey A: Seller registers a motorcycle

1. Seller logs in.
2. Opens Add Vehicle.
3. Chooses Petrol Motorcycle.
4. Enters common and category-specific fields.
5. Submits.
6. Vehicle appears in My Vehicles.
7. Refresh confirms persistence.

Journey B: Seller uploads service records

1. Seller opens Documents.
2. Selects the motorcycle.
3. Uploads a service invoice.
4. Upload/scan status is shown.
5. OCR extracts visible details.
6. Seller reviews fields.
7. Mileage event is saved with source reference.
8. Passport timeline updates.

Journey C: Mileage inconsistency

1. A historical record contains 65,000 km.
2. A later record contains 42,000 km.
3. Deterministic chronology rule flags the decrease.

4. Finding includes both records and their dates.
5. Seller can submit clarification/evidence.
6. Buyer sees the permitted finding and explanation.
7. The system does not automatically call it confirmed odometer fraud.

Journey D: Damage inspection

1. Seller uploads a damage photo.
2. Image validation and scanning occur where configured.
3. YOLO job runs only if the model/adapter is available.
4. Seller reviews any model suggestion.
5. Confirmed damage record is saved with evidence.
6. Supported 3D viewer displays the component hotspot.
7. Buyer sees the permitted finding.

Journey E: Buyer marketplace flow

1. Buyer logs in.
2. Searches for a scooter.
3. Applies category and price filters.
4. Opens listing.
5. Reviews passport summary.
6. Saves vehicle.
7. Sends inquiry.
8. Seller receives an in-app notification.
9. Seller replies.
10. Buyer sees conversation update.

Journey F: Access isolation

1. Seller A creates a vehicle and private document.
2. Seller B attempts to access Seller A's vehicle/document.
3. Backend rejects unauthorized access.
4. Buyer requests private raw document URL.
5. Backend denies access unless a permitted disclosure exists.
6. Audit/security log records the denied event where configured.

19.3 Testing rules

- Use an isolated test database.
- Use synthetic fixtures clearly labelled as synthetic.
- Never run destructive tests against production data.
- Exercise the UI; do not mark a feature passed only because an endpoint exists.
- Verify API response and database persistence.
- Refresh the page and verify persisted state.
- Capture screenshots and Playwright traces for failures.
- Re-run tests after fixes.
- Distinguish `PASS`, `FAIL`, `BLOCKED`, `NOT IMPLEMENTED`, `DEMO ONLY` and `EXPERIMENTAL`.

20. Feature Status and Known Limitations

This section reflects the latest honest status supplied by the project owner in the conversation. It is not a new execution of the repository or deployment.

20.1 BLOCKED - external credentials or infrastructure

Feature	Current reported issue	What is required
S3/object storage	Not configured	Bucket, endpoint, credentials and upload integration
SMTP/SMS	Not configured	Verified sender/provider account and credentials
ClamAV/external AV	Not connected	Running scanner and upload-pipeline integration
VAHAN/government registry	No authorized credentials/adapter	Approved access, consent and adapter
Insurer/theft registry	No provider access	Authorized provider and integration
PostgreSQL split deployment	Not provisioned/configured	Managed DB, migrations, connectivity and tests
Docker locally	Local Docker daemon unavailable	Working daemon or CI build runner
Playwright locally	Browser binaries not provisioned	Browser installation and OS dependencies

20.2 NOT IMPLEMENTED

Feature	Current reported issue	Next step
Blockchain anchoring	Only local SHA-256 chain	Implement and test real network anchoring if required
Licensed non-car 3D GLBs	Cars have labelled generic GLB; two-wheelers have schematic	Obtain/prepare licensed category-specific assets and component mappings

20.3 DEMO ONLY / EXPERIMENTAL

Feature	Current reported issue	Next step
XGBoost	Synthetic-data training only	Obtain representative labelled data and validate model
PaddleOCR	Not installed; RapidOCR reported active	Compare provider only if needed; keep working RapidOCR path
LLM narrative	`TD_LLM_PROVIDER=none`	Configure supported provider or retain deterministic reports

20.4 Important interpretation

- RapidOCR may be a working OCR provider even while PaddleOCR is not installed. Test actual upload-to-extraction flow before marking OCR unavailable.
- Deterministic verification rules can be functional without an LLM.
- A local SHA-256 hash chain provides local integrity evidence, not blockchain anchoring.
- A generic GLB is not an exact vehicle model or vehicle-specific scan.

- A YOLO adapter or weights file does not prove the upload-to-detection-to-review-to-database workflow is complete.
- External government/insurance results must not be simulated or fabricated.

21. Implementation Roadmap

Phase 1 - Secure and stable foundation

- Disable anonymous workspace fallback in production.
- Keep demo seeding disabled in production.
- Verify role-based access and cross-user isolation.
- Confirm persistent database and file storage.
- Establish backups and safe migration procedures.

Phase 2 - Core vehicle lifecycle

- Complete vehicle registration for priority categories.
- Complete document upload and review.
- Complete history timeline and mileage records.
- Complete marketplace listing and inquiry workflows.
- Ensure Buyer and Seller screens use real persisted data.

Phase 3 - Verification and intelligence

- Complete RapidOCR upload-to-result workflow.
- Complete deterministic document/identity/mileage checks.
- Complete evidence linking and review lifecycle.
- Complete YOLO flow only when suitable weights exist.
- Label XGBoost accurately until real validation data exists.
- Add LLM narrative only as an optional evidence summarizer.

Phase 4 - 3D inspection

- Add licensed motorcycle and scooter assets.
- Create category-specific model fallbacks.
- Map actual meshes to component IDs.
- Connect damage records and hotspots.
- Enable exploded/X-Ray only for supported assets.

Phase 5 - Deployment and quality

- Provision PostgreSQL.
- Configure persistent object storage.
- Complete environment and health checks.

- Provision Playwright in CI.
- Build Docker images in CI.
- Complete full Seller-to-Buyer browser journeys.

Phase 6 - Optional external integrations

- Authorized vehicle registry adapters.
- Insurance/theft providers.
- Email/SMS delivery.
- External malware scanning.
- Blockchain testnet anchoring.
- Production ML dataset and validation.

22. Definition of Done

A TrustDrive feature is considered complete only when all applicable conditions are met:

1. The frontend page and action are usable.
2. The API request reaches the intended backend handler.
3. Authentication and authorization are enforced.
4. Server-side validation is performed.
5. Business logic produces the expected result.
6. Database or object storage operation succeeds.
7. The UI displays the actual server result.
8. Refresh/re-login confirms persistence.
9. Errors and unavailable providers are shown honestly.
10. Automated tests cover normal, invalid and unauthorized cases.
11. End-to-end browser test passes where applicable.
12. Documentation and feature status are updated.

Release readiness checklist

- [ ] Buyer authentication and dashboard tested.
- [ ] Seller authentication and dashboard tested.
- [ ] Admin routes protected.
- [ ] Vehicle registration tested per supported category.
- [ ] Document upload and secure retrieval tested.
- [ ] OCR workflow tested with real fixtures.
- [ ] Mileage and identity checks tested.
- [ ] Vehicle history persistence tested.

- [ ] Damage review workflow tested.
- [ ] 3D model mapping and capabilities tested.
- [ ] Listing lifecycle tested.
- [ ] Buyer inquiry and seller response tested.
- [ ] Role and ownership isolation tested.
- [ ] Database backup/migration path checked.
- [ ] Production secrets protected.
- [ ] Demo seeding and anonymous access disabled in production.
- [ ] Render build and health checks pass.
- [ ] Playwright E2E suite passes in CI.
- [ ] Remaining blocked features are disclosed.

23. Glossary

Term	Meaning
Vehicle History Passport	Centralized, evidence-linked record of available vehicle information and findings
OCR	Optical Character Recognition; converts text in images/PDFs into machine-readable text
Evidence correlation	Comparing linked records to find agreement, conflict or missing information
Deterministic rule	A repeatable check based on explicit logic
YOLO	Object-detection model family; vehicle-damage use requires appropriate weights and validation
XGBoost	Gradient-boosted decision tree library used for supervised ML
LLM	Large language model, suitable for summarizing evidence but not inventing facts
LangGraph	Workflow orchestration framework for stateful agent/tool processes
GLB/glTF	3D asset formats used for meshes, materials, textures and scene data
Raycasting	Technique used to detect which 3D object a user points to or clicks
Exploded view	A visualization that separates real model components to show their arrangement
X-Ray mode	A rendering mode that makes selected/all model materials transparent
SHA-256	Cryptographic hash used to identify file-byte changes
RBAC	Role-Based Access Control
E2E	End-to-end testing through a complete user workflow
Idempotency	Repeating an operation safely without creating unintended duplicates

Final Product Statement

TrustDrive turns fragmented vehicle records into an organized, evidence-based vehicle investigation. It combines document extraction, deterministic consistency checks, AI-assisted analysis, vehicle history, category-aware inspection and a buyer/seller marketplace. Its reliability depends on traceable evidence, secure data handling, honest status labels and real end-to-end testing.

The long-term objective is to support safer and more transparent used-vehicle decisions across cars, motorcycles, scooters, electric vehicles and other vehicle categories.